Atlanta Cloud + AI Conference 2026

Shift-Left Performance Engineering

Implementing Performance Signatures in CI/CD Pipelines

What if every commit carried a performance verdict, automatically?

Kandasamy Selvaraj

Performance Defects Are Expensive.

We Keep Finding Them Too Late.

1× to fix in development.
60 to 100× to fix in production.
Source: CISQ, 2022

Cost multiplier when defects reach production vs. development.

- Testing happens post-feature. Debugging context of code changes is already gone.
- Specialized teams support and dependency that last hours or days to get feedback
- Agentic AI is probabilistic, not deterministic, without guardrails, failures can cascade
- Pass/fail relies on manual, subjective interpretation.
- Microservices make late-stage testing unpredictable.
- Developers get no continuous feedback loop on performance.

Development	Integration	System Test	Pre-Production	Production
Low cost to fix	Rising cost	Higher cost	Very high cost	Extremely high

Every Commit Carries a Performance Verdict.

Here is how we make that happen automatically.

BEFORE

- Performance tested once, late in the cycle.
- Manual expert reviews creating team bottlenecks.
- Static thresholds that go stale after each release and fiscal year.
- Feedback arrives days after the code was written.

AFTER

- Performance validated at every code commit.
- Automated statistical gates. Zero specialist needed.
- AI-learned baselines that evolve with the system.
- Actionable feedback within 15 to 20 minutes.

This framework sits in the Agile/DevOps space: continuous, automated, and invisible to the developer once set up.

Feedback in 15 to 20 minutes. Self-service. Zero specialized expertise required.

Six Layers. One Pipeline. No Manual Gates.

A production-validated architecture where performance is enforced, not reviewed.

1

Developer Self-Service

Git-triggered pipelines, parameterized test scripts, 15-minute zero-expertise setup.

2

Pipeline Orchestration

Declarative DSL, performance signature gates at 99.5% accuracy, parallel execution.

3

Load Generation

JMeter 10 to 1K users, Kubernetes auto-scaling, real-time metrics streaming.

4

Observability Engine

Inject build context into traces. Your APM tool correlates commit to bottleneck automatically.

5

Data Persistence

30-day statistical baseline, trend analysis, adaptive threshold calibration.

6

AI Decision Layer

ML-powered go/no-go gate. Anomaly detection replaces static thresholds. Autonomous and explainable.

NEW

From Subjective Thresholds to Objective Statistical Performance Signature

$$\text{Deviation Score} = \frac{|\ \text{Current p90} - \text{Baseline p90}\ |}{\text{Baseline Standard Deviation}}$$

50ms increase with 20ms σ = 2.5σ deviation **FLAGGED.**

Baselines built from 30 days of history, minimum 10 successful runs, using mean and standard deviation per metric.

Objectivity
Mathematically defined pass/fail.

Adaptability
Baselines evolve as the app improves.

Sensitivity
Detects small but statistically significant regressions.

Transparency
Developers intuitively understand standard deviations.

The static threshold tells you whether a number is acceptable.
The deviation score tells you whether a number is normal.

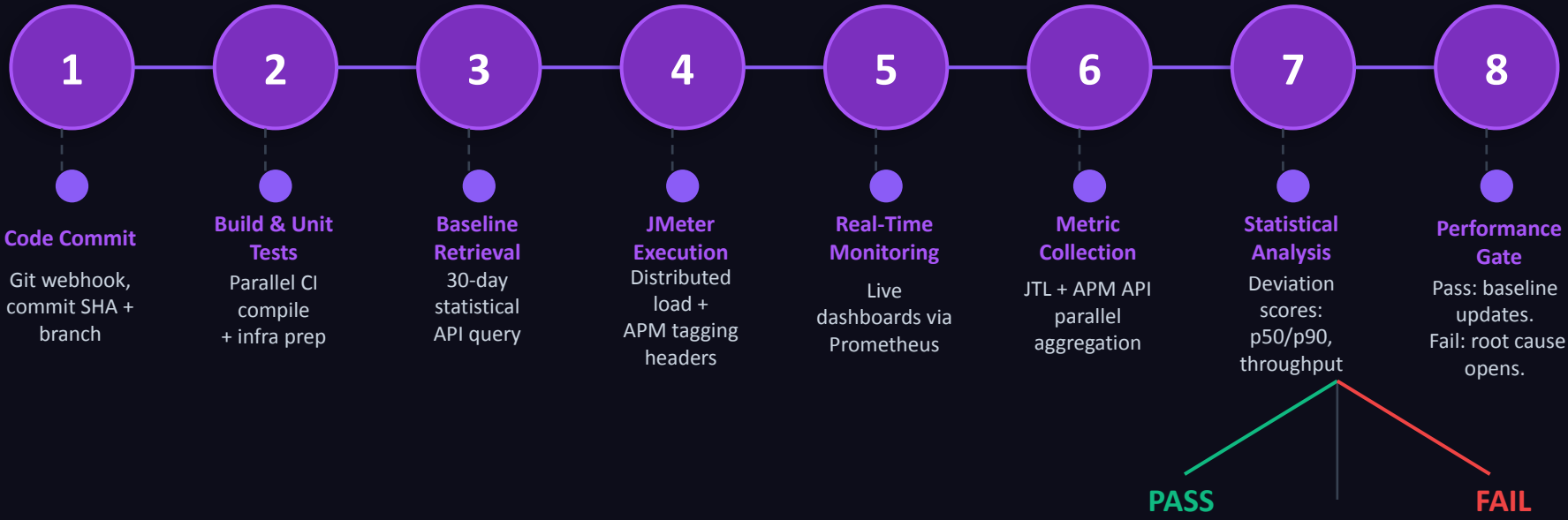
Multi-Dimensional Performance Gate. No Guesswork.

Every dimension must pass. Break one and the build stops — with a reason.

METRIC CATEGORY	SPECIFIC METRICS	THRESHOLD
Response Time	p50, p90 latency	±2σ
Throughput	Requests per second	±2σ
Error Rate	HTTP errors, exceptions	**+1% absolute**
Resource Utilization	CPU, memory, network	±3σ
Database Performance	Query time, connection count	±2σ

All five dimensions must pass. Your observability tool surfaces the why — not just the what.

Code Commit to Performance Verdict in 15 to 20 Minutes



In an Agentic AI pipeline, Steps 3 to 8 run autonomously.

The AI agent retrieves baselines, executes load, collects metrics, scores deviations, and issues a go/no-go with a plain-English explanation. No human in the loop.

Three Integration Patterns That Work With Any Stack

The tools are examples. The patterns are what matter.

Orchestration Pattern

Jenkins ↔ JMeter

Pipeline triggers load execution, parses results, and enforces performance gates on every build.

Observability Pattern

JMeter ↔ Your APM Tool

Inject build context into every request. Your observability layer correlates commit to bottleneck automatically.

Visibility Pattern

JMeter ↔ Grafana

Real-time metrics during the test. 90-day trend history across builds so teams see performance evolving over time.

```
// Inject build context into every load test request — works with i.e Dynatrace, Datadog, New Relic, or any APM
  httpSampler.addNonEncodedArgument("X-Build-ID",    "${BUILD_ID}",    "");
  httpSampler.addNonEncodedArgument("X-Commit-SHA",  "${GIT_COMMIT}",  "");
  httpSampler.addNonEncodedArgument("X-Branch",      "${BRANCH_NAME}", "");
  // Your APM now links every trace back to the exact commit that triggered it
```

Tag your traffic. Any APM that reads HTTP headers can now trace a performance problem back to the exact commit.

How We Did It: 8 Weeks.

A real microservices system. No shortcuts.

SYSTEM UNDER TEST

- 50+ microservices: Java Spring Boot + React
- Kubernetes on Cloud
- RDS + Redis Cache + Event Streaming Platform
- ~ Around 50,000 - 80,000 requests/minute peak load

8-WEEK IMPLEMENTATION TIMELINE

Week 1 ← *Start here today*
IaC provisioning + AI-assisted config generation for Jenkins, Prometheus, Grafana, your APM tool.

Weeks 2 to 3
LLM generates JMeter scripts from OpenAPI specs and production traffic logs.

Weeks 4 to 5
Declarative DSL, performance signature gates, adaptive baseline tuning.

Week 6
Top 5 critical workflows with intelligent gate validation.

Weeks 7 to 8
AI-generated runbooks, self-service templates, team onboarding.

How we measured it:

We tracked the same metrics for 6 months before rollout, then 6 months after. Same system, same conditions, no cherry-picking.

Performance Problems Caught Earlier.

Fewer Reaching Production.

	BEFORE	AFTER	CHANGE
Defects Found in Development	18%	58%	+222%
Defects Found in Test	35%	31%	-11.4%
MTTD (hours)	52.3	0.4	-99.2%

MTTD: 52 hours → 24 minutes.

Immediate automated feedback fundamentally changes operational efficiency.

Beyond Numbers: Culture and Developer Experience

91%

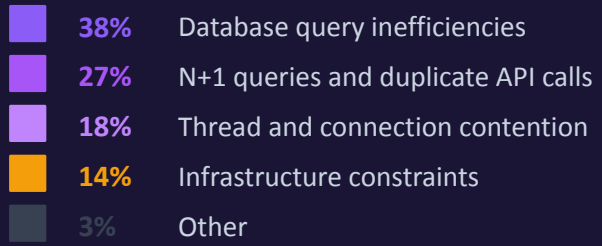
of failures came with an automated root cause.

Nobody had to investigate manually. The system told them where to look.

Self Service Model

- Increased confidence in their own code performance
- Faster feature delivery and reduced incidents
- Improve overall application Performance and SLA/SLO compliance

ROOT CAUSE BREAKDOWN (APM Tool)



When developers trust the feedback loop, they stop avoiding performance. That cultural shift showed up in the data: 30% faster delivery and 60% fewer escalations.

Honest Constraints and Lessons Learned

No framework is perfect. Here is what we learned the hard way.

Key Limitations

Cold start takes 5 to 10 runs.

The AI needs data before it can gate reliably. We hardcoded manual thresholds for the first sprint. Plan for it.

18 minutes adds up for fast teams.

High-frequency commit teams felt the pipeline drag. We moved performance gates to nightly for those branches.

Training still takes 6 to 8 hours.

Self-service templates helped but did not replace onboarding. Budget a interactive 2 to 3 hour session per team, not an email.

Proven Best Practices

Start with your highest-risk journeys.

Pick the 5 workflows that would cost the most if they regressed. Prove value there first, then expand.

Run 10 successful tests before enabling gates.

Gates armed too early produce noise. Patience in week 1,2 saves frustration in weeks 3 through 8.

Celebrate developers who catch their own regressions.

Culture follows incentives. Public credit for self-caught issues changed behavior faster than any tool.

The Foundation for Next-Generation Performance Engineering

-30 to 40%	-44%	-30%	98.9%	Days → Minutes
Performance Issues	Response Time	Release Cycle	Gate Accuracy	MTTD

Statistical baseline comparison transforms performance gates from subjective manual interpretation to objective, reproducible, automated decisions — making performance a natural part of every developer's workflow.

AI-Enhanced Test Generation

ML-based test plan synthesis from production traffic.

Predictive Performance Analysis

Predict regression risk before execution runs.

Open Telemetry Standardization

Replace proprietary instrumentation with vendor-neutral tracing.

Shift-Right Integration

Synthetic monitoring and progressive delivery with automated rollback.

"As software architectures grow increasingly complex, automated continuous performance validation becomes essential rather than optional."

Fast Feedback Is Not the Same as Full Validation.

Pipeline performance testing and full-scale performance engineering are not competitors. They are partners. You need both.

What Shift-Left Pipeline Testing Gives You

- Catches regressions at commit time before they compound.
- Developer feedback in 15 to 20 minutes per build.
- Runs lightweight in controlled environments like DIT and FIT.
- Scoped to a single change — not the full platform.
- Empowers developers to own their own performance quality.

What Your Performance Engineering Team Validates

- Stress tests at 10× or more of production load — not possible in pipeline environments.
- Soak testing over hours or days to expose memory leaks and gradual degradation.
- Full production data volumes, topology, and real-world service interdependencies.
- Regulated industry sign-off — financial services, healthcare, and compliance-driven capacity evidence.
- End-to-end business journey simulation across all services simultaneously.

Full-scale performance engineering validates the whole system under real-world conditions.

Full-scale performance engineering validates what only emerges under production-scale stress, sustained load, and compound system behaviour. The performance engineering team owns the latter. Developers own the former.

Your pipeline has no memory.

Fix that this week.

1 **Start Week 1 today.**

Pick your top 3 highest-risk user journeys. Everything else can wait.

2 **Tag your traffic.**

Add X-Build-ID and X-Commit-SHA headers to your load tests. Any observability tool reads them.

3 **Set one gate.**

Response time p90 ±2σ. Just one. Ship it. Add the rest next sprint.

linkedin.com/in/kandasamy-selvaraj-3ab97835/

Resources and templates shared after the talk.
https://github.com/kandasamyperf-stack/perf-grimoire

ATL CLOUD + AI CON

Sponsor

Platinum Sponsor

ASSURANT®

FULTON COUNTY SCHOOLS
INNOVATION
ACADEMY

improving
It's what we do. ™

Swag

Frontend Queens